

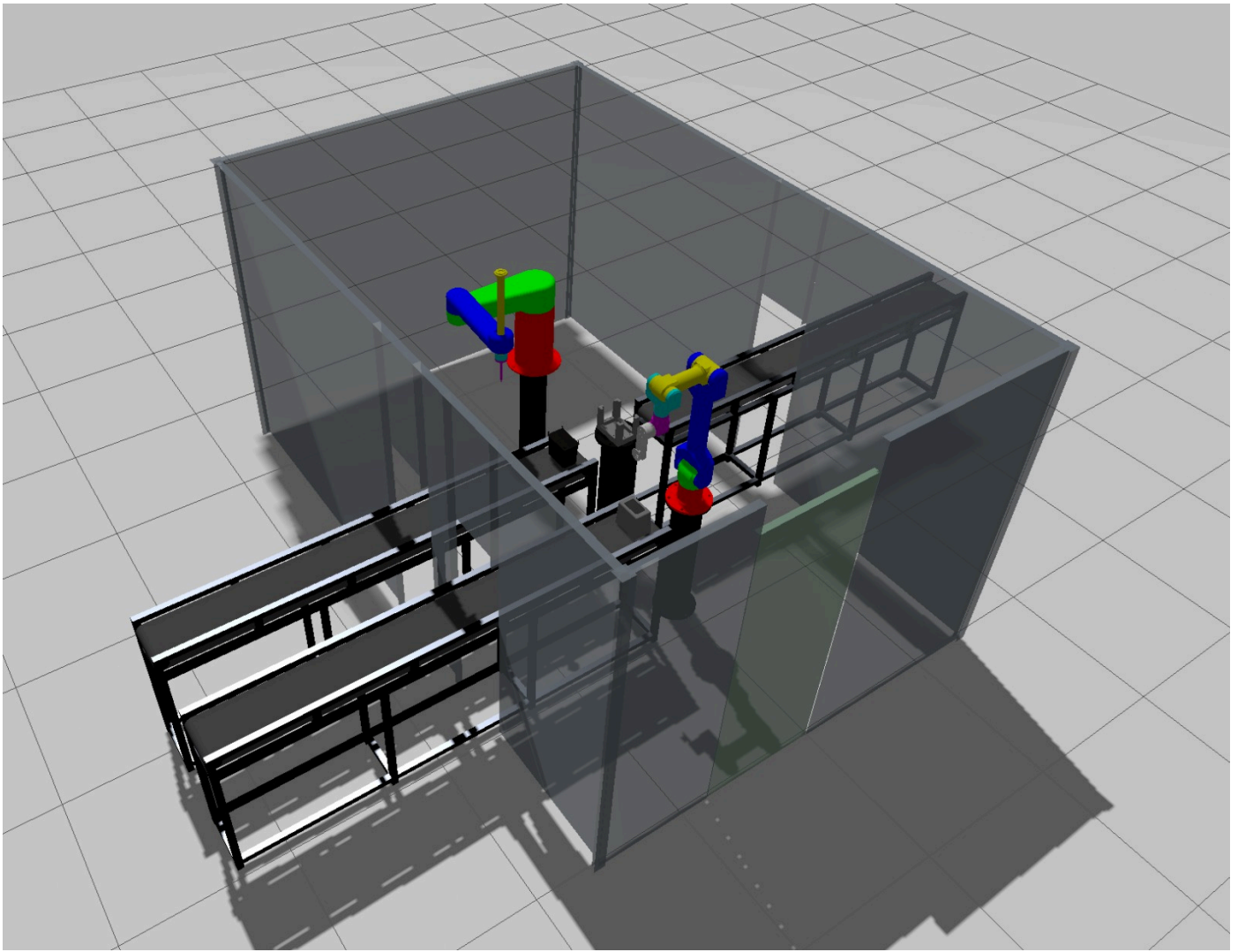
- ROD – Robot Cell Simulation
 - Übersicht
 - Packages
 - Setup
 - Voraussetzungen
 - Repository klonen
 - Workspace bauen
 - Starten
 - HMI – Bedienung
 - Headerzeile
 - TCP-Steuerung
 - PICK / PLACE / SCREW / Scene Reset
 - Gelenkwinkel (live)
 - Pose-Verwaltung
 - Sequenz
 - CSV Import / Export
 - Log-Panel
 - Multi-Object Pick
 - Technische Details
 - Pick-Implementierung
 - IK-Konfiguration
 - Roboter-Weltrahmen-Offsets
 - Scene Reset
 - Status & TODOs

ROD – Robot Cell Simulation

ROS2 Jazzy | Gazebo Harmonic | MoveIt2

FH Technikum Wien – Robotics Engineering [GitHub Link](#)

Übersicht



Simulation einer industriellen Roboterzelle mit zwei Robotern:

- **6-DOF Knickarm** (kinematisch angelehnt an UR15) mit Saugnapf-Endeffektor
- **SCARA** (4-DOF) mit Schraubwerkzeug

Usecase: Toaster-Gehäuse-Montage

1. Knickarm greift Gehäuse (`toaster_shell`) vom Förderband → legt in Fixiereinheit
2. Knickarm greift Deckel (`toaster_innen`) + 4 Schrauben gleichzeitig → legt auf Gehäuse
3. SCARA verschraubt Deckel (4 Schrauben einzeln)
4. Knickarm greift fertige Assembly (alle 6 Teile) → transferiert auf Ausgangs-Förderband

Packages

Package	Beschreibung
<code>robot_arm_6dof_assembly</code>	URDF, Meshes, ros2_control Config für den 6-DOF Knickarm
<code>arm_moveit</code>	Movelt2 Konfiguration für den Knickarm (SRDF, Kinematics, Controller)
<code>scara_4</code>	URDF, Meshes, ros2_control Config für den SCARA
<code>scara_moveit</code>	Movelt2 Konfiguration für den SCARA
<code>rod_scene</code>	Szenenobjekte als GLB-Meshes (Säulen, Fixiereinheit, Förderbänder, Schrauben)
<code>rod_cell</code>	Cell Launch File — startet Gazebo, beide Roboter und alle Szenenobjekte
<code>rod_hmi</code>	Dear ImGui HMI — TCP-Steuerung, Pose-Verwaltung, Sequenz, CSV Import/Export
<code>SolidWorks</code>	Originale CAD-Dateien (COLCON_IGNORE, nur Referenz)

Setup

Voraussetzungen

```
sudo apt install ros-jazzy-moveit ros-jazzy-ros-gz ros-jazzy-gz-ros2-control \
ros-jazzy-ros2-control ros-jazzy-ros2-controllers ros-jazzy-joint-state-
publisher* \
libglfw3-dev libglfw3
```

Repository klonen

```
mkdir -p ~/rod_ws/src
cd ~/rod_ws/src
git clone https://github.com/Raini20/ROD.git .
git checkout pick_place
```

Workspace bauen

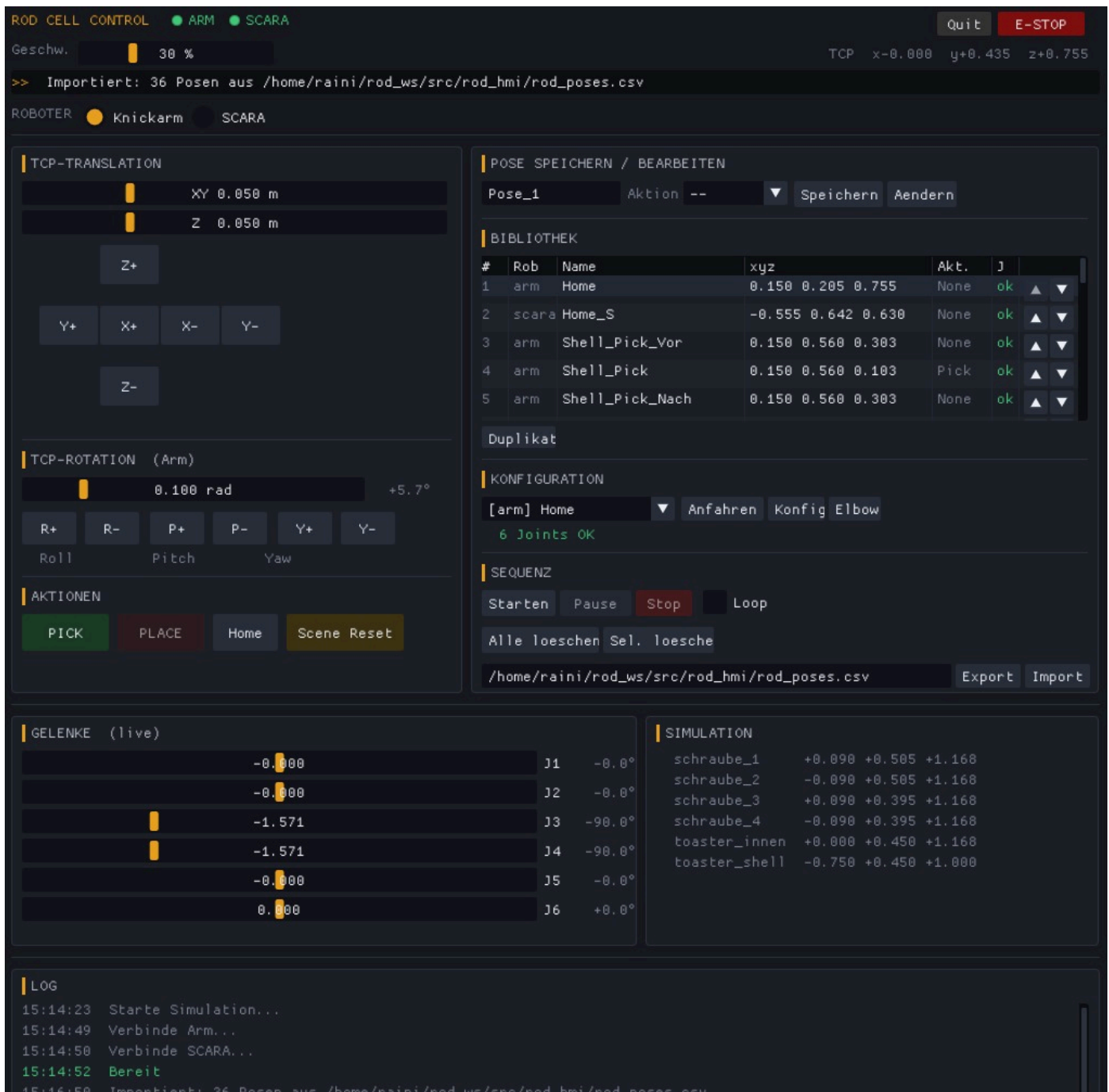
```
cd ~/rod_ws && colcon build && source install/setup.bash
```

Starten

Ein einziger Befehl öffnet das GUI und startet Gazebo + beide MoveGroups automatisch im Hintergrund (~25 s Wartezeit).

```
cd ~/rod_ws && source install/setup.bash  
ros2 run rod_hmi rod_hmi
```

HMI – Bedienung



Das HMI (`rod_hmi`) ist die primäre Schnittstelle zur Simulation.

Headerzeile

- **E-STOP** (rot, oben rechts) — sofortiger Nothalt, stoppt beide Roboter via `mg->stop()`. Alle Bewegungsbuttons werden deaktiviert. Mit **"E-Stop Reset"** wird der Zustand zurückgesetzt.
- **Quit** (grau, rechts neben E-STOP) — beendet das HMI sauber: stoppt zuerst alle Hintergrundprozesse (Gazebo, beide MoveGroups, `ros2_control`, `robot_state_publisher`) per `SIGTERM`, wartet kurz, schickt `SIGKILL` an hartnäckige Prozesse und setzt den ROS2 Daemon zurück — damit der nächste Start eine saubere Umgebung vorfindet.

- **Geschwindigkeit** (Slider, 5–100 %) — Geschwindigkeits-Override für alle Bewegungen beider Roboter, wird live angewendet.
- **TCP-Anzeige** (rechts) — aktuelle TCP-Position des gewählten Roboters in Weltkoordinaten.
- **Statuszeile** — letzte HMI-Meldung; vollständiges Log im Log-Panel unten.

TCP-Steuerung

- X+/X-/Y+/Y-/Z+/Z- Buttons für kartesische Bewegung
- Schrittweite per Slider einstellbar (1 mm - 200 mm), separat für XY und Z
- TCP Rotation Roll/Pitch/Yaw (nur Knickarm), Schrittweite in Rad + Grad angezeigt
- **Home** — fährt den gewählten Roboter in die im SRDF definierte **home**-Pose

PICK / PLACE / SCREW / Scene Reset

Manuelle Buttons zum direkten Auslösen einer Aktion.

- **PICK** (Knickarm) — greift alle Objekte innerhalb von 300 mm Radius gleichzeitig
- **PLACE** (Knickarm) — setzt alle gegriffenen Objekte ab
- **SCREW** (SCARA) — greift die nächste Schraube (bevorzugt **schraube_***, exakt 1 Objekt), fährt 25 mm runter & dreht J4 um 3 Umdrehungen, setzt ab, fährt zurück
- **Scene Reset** — setzt alle Simulationsobjekte auf ihre exakten Startpositionen und -orientierungen zurück

Gelenkwinkel (live)

Slider zeigen die **aktuelle Gelenkposition** in Echtzeit (Aktualisierung alle 200 ms). Slider ziehen und **loslassen** → Roboter fährt direkt auf diesen Winkel, ohne extra Button.

Winkelanzeige: Radiant im Slider + Grad als Label (z. B. **-80,3°**).

J3 des SCARA (Linearachse) wird in mm angezeigt (-400 mm bis 0 mm).

Pose-Verwaltung

Speichern:

Name eingeben → Aktion wählen → **"Speichern"**

TCP-Pose und Gelenkkonfiguration werden automatisch gemeinsam gespeichert.

Der Namensvorschlag zählt automatisch hoch (z. B. **Pose_1** → **Pose_2**).

Bearbeiten:

Tabellenzeile anklicken → Name und Aktion werden in die Felder übernommen.

"Ändern" — aktualisiert Name/Aktion der gewählten Pose ohne die Position neu zu teachen.

Aktionen im Dropdown:

Roboter	Optionen
Knickarm	--, Pick , Place , Reset
SCARA	--, Screw , Reset

Reset führt einen Scene Reset als Teil der Sequenz aus — sinnvoll als letzter Schritt im Loop.

Gespeicherte Posen Tabelle:

Zeigt alle Posen mit Index, Roboter, Name, Position, Aktion und Konfigurations-Status (**ok** / **-**).

Zeile anklicken → Pose wird in den Feldern und im Dropdown ausgewählt.

↑ / ↓ — Reihenfolge der Einträge ändern.

Duplikat — kloniert die gewählte Pose mit **_2**-Suffix.

Konfiguration:

- **Anfahren** — fährt die gewählte Pose an (bevorzugt gespeicherte Joint-Werte)
- **Konfig** — speichert die aktuelle Gelenkkonfiguration für diese Pose nach
- **Elbow** — spiegelt Joint 3 um 180° (Elbow-Up / Elbow-Down) ⚠ *experimentell*

Grünes **ok** = Joint-Werte gespeichert → Sequenz verwendet diese exakte Konfiguration.

Sequenz

Button	Funktion
Starten	Führt alle Posen der Reihe nach aus
Pause / Resume	Hält nach der aktuellen Bewegung an, setzt fort
Stop	Bricht die Sequenz ab und stoppt den Roboter sofort
Loop	Wiederholt die Sequenz endlos (Zähler rechts daneben)

Der aktuell ausgeführte Schritt wird in der Tabelle amber hinterlegt.

Pro Schritt:

1. Gespeicherte Joint-Werte werden bevorzugt (deterministische Konfiguration)
2. Falls keine vorhanden → kartesischer Pfad (fraction > 0,5)
3. Falls auch das scheitert → MoveIt Pose-Planner als Fallback

Loop + Scene Reset: **Reset**-Aktion ans Ende der Sequenz hängen → nach jedem Durchlauf werden alle Objekte in die Ausgangslage zurückgesetzt.

CSV Import / Export

Format:

```
# ROD Pose Export
# robot,name,x,y,z,qx,qy,qz,qw,action,j0,j1,...
arm,Home,0.150,0.205,0.755,0.707,0.0,0.707,0.0,None,-0.44,-0.04,-1.53,-1.57,2.70,-1.57
```

- Pfad im Textfeld einstellbar (Standard: `~/rod_ws/src/rod_hmi/rod_poses.csv`)
- **Export** — schreibt alle aktuellen Posen inkl. Joint-Werte
- **Import** — lädt Posen aus CSV, hängt sie an die bestehende Liste an
- Rückwärtskompatibel: alte CSVs ohne Joint-Spalten werden korrekt eingelesen

Log-Panel

Scrollbares Protokoll aller Statusmeldungen mit Timestamp (**HH:MM:SS**).

Farbkodierung: Rot = Fehler, Grün = Erfolg, Grau = Info.

Scrollt automatisch auf den neuesten Eintrag.

Multi-Object Pick

Der Pick-Mechanismus greift automatisch alle Objekte innerhalb eines **350 mm Radius** um den TCP:

Schritt	Gegriffene Objekte
Shell holen	<code>toaster_shell</code> (1 Objekt)
Deckel + Schrauben holen	<code>toaster_innen</code> + 4 × <code>schraube_*</code> (5 Objekte)
Assembly transferieren	alle 6 Teile

Alle Objekte folgen dem TCP mit korrekter relativer Position und Orientierung.

Der SCARA-SCREW greift **exakt 1 Objekt** (nächste Schraube im 300 mm Radius, bevorzugt `schraube_*`).

Technische Details

Pick-Implementierung

`do_pick_impl(mg, base_ox, base_oy, base_oz, prefer_substr, max_count):`

- `prefer_substr` — Objekte deren Name diesen String enthält werden bevorzugt
- `max_count` — maximale Anzahl zu greifender Objekte (Arm: 999, SCARA-Screw: 1)
- Objekte werden nach Präferenz, dann nach Distanz sortiert

IK-Konfiguration

`computeCartesianPath` allein wählt Elbow-Konfiguration frei → inkonsistente Bewegungen.

Fix: Posen werden mit `getCurrentJointValues()` gespeichert. Die Sequenz setzt `setJointValueTarget()` mit diesen Werten → deterministische Wiederholung.

Roboter-Weltrahmen-Offsets

Aus `cell.launch.py` (`-x -y -z` Spawn-Argumente):

Roboter	X	Y	Z
Knickarm	-0,75 m	0,00 m	1,00 m
SCARA	+1,00 m	0,00 m	1,00 m

Scene Reset

Stellt **Position und Orientierung** aller Objekte aus `k_init_pos` / `k_init_ori` wieder her.

Status & TODOs

- ☒ 6-DOF Knickarm URDF + SCARA URDF mit Endeffektoren ⚠ *TCP-Offset (Saugnapf) in Yaw-Achse nicht korrekt*
- ☒ MoveIt2 Konfiguration für beide Roboter
- ☒ Gazebo Harmonic — beide Roboter in einer Simulation, namespaced Controller
- ☒ Zellszene: Säulen, Fixiereinheit, Förderbänder, Toasterteile, 4 Schrauben, Schutzzaun
- ☒ HMI startet alles automatisch (~25 s)
- ☒ TCP-Steuerung (Translation + Rotation)
- ☒ Gelenkwinkel live mit Direktsteuerung (Slider loslassen → Roboter fährt)
- ☒ Gelenkwinkel-Anzeige in Grad und Radiant
- ☒ J3 SCARA (Linearachse) Anzeige in mm
- ☒ Pose speichern mit automatischer Joint-Konfiguration + Auto-Nummerierung
- ☒ Pose bearbeiten (Name/Aktion ohne Neu-Teachen)
- ☒ Pose duplizieren und Reihenfolge ändern (↑↓)
- ☒ Konfigurationsauswahl mit Anfahren / Konfig / Elbow Flip
- ☒ Klickbare Pose-Tabelle mit Sequenz-Highlighting
- ☒ Sequenz mit Starten / Pause / Resume / Stop / Loop
- ☒ Loop-Modus mit Scene-Reset-Aktion für saubere Wiederholung

- ☒ Multi-Object Pick (bis zu 6 Objekte gleichzeitig, radius-basiert)
- ☒ SCARA Schrauben-Sequenz (Single Pick → 25 mm runter & J4 3× drehen → absetzen → zurück)
- ☒ PICK / PLACE / SCREW / Home / Scene Reset manuelle Buttons
- ☒ E-STOP mit sofortigem `mg->stop()` auf beiden Robotern
- ☒ Geschwindigkeits-Override (5–100 %) live anwendbar
- ☒ CSV Import + Export mit Joint-Werten, konfigurierbarer Pfad
- ☒ Zeitgestempeltes Log-Panel mit Farbkodierung
- ☒ Quit-Button mit sauberem Shutdown
- ☒ Dokumentation als PDF
- ☒ Backup-Video der Simulation
- ☒ Pick/Place — Greifer-Simulation aktiv (visuelle Verfolgung via Gazebo `set_pose`), kein physischer `link_attacher`
- ☒ Screw — SCARA J4-Rotation animiert, aber keine Physik-Interaktion mit der Schraube